Homework 2 (CS59300 ASE)

Test Generation and Improvement Using Coding Agents

Weixi Tong November 2, 2026

I used each test run, together with Codex's review, to decide what to ask for next. The initial results pointed to missing PDF cases. After those cases were added, the remaining gaps pointed to service handling and weak checks of saved results. This report explains how those observations shaped the two improvement prompts and how I judged their results.

1. Test Generation Approach

Papertrail, my Homework 1 application, lets users search arXiv, save papers, upload PDFs, generate summaries, and ask questions about a paper. It uses Flask, SQLite, and pypdf, with a JavaScript frontend. I began with a broad request to read the application and its requirements, then write tests for its main functions:

> Generate meaningful, executable tests for the main functionality, including normal cases, edge cases, and error handling. Prefer the existing unittest framework unless there is a clear reason to change it.

This gave me a starting point for finding out where more guidance was needed. Codex used `unittest` and the Flask test client, producing 37 methods.

Test group	Methods	Examples
Main application functions	15	Search, saving papers, uploads, summaries, questions, editing, and deletion.
Input checks and errors	14	Invalid requests, bad PDFs, missing papers, service failures, and login checks.
Individual functions	8	Database transactions, text selection, citations, and service requests.

The tests used temporary databases and generated sample PDFs. Calls to arXiv and the model used fixed sample responses. This made it possible to compare runs without depending on a live service, while still testing the application's handling of those responses.

I preserved the initial tests and results before asking for changes. I also kept the application code the same throughout the experiment, so changes in coverage would reflect changes to the tests. All stages measured the same four Python files with coverage.py 7.17.0: `app.py`, `database.py`, `pdf_tools.py`, and `providers.py`.

Codex could inspect existing tests, but the 14 Homework 1 tests and older HW2 experiments were excluded from the reported totals. The comparison uses the three versions in `initial_stage`, `round1_stage`, and `round2_stage`. [1–4]

2. Initial Test Evaluation

The first run passed 35 of 37 methods and covered 474/522 executable lines (90.80%) and 135/174 branch outcomes (77.59%). Its saved output ended with:

```
Ran 37 tests in 0.563s

FAILED (failures=3)
```

Before deciding on the next prompt, I needed to understand the failures. Two methods failed on three input cases; none failed because of a setup error. Codex's review linked them to two application bugs.

Bug 1: invalid analysis input caused a server error. An analysis request with `{"kind": []}` or `{"kind": {}}` returned HTTP 500 rather than an input error 400. The app checked the value in a set before checking its type. The detecting method in `test_validation.py` was:

```
test_invalid_analysis_payloads_are_client_errors_without_provider_calls
```

Bug 2: an unusual page number caused a server error. The search value `page=%C2%B2` contains the superscript character ². It passed `isdigit()`, but `int()` could not convert it. The app again returned 500 rather than 400. The detecting method was:

```
test_invalid_search_parameters_are_client_errors
```

The expected 400 responses followed how the app handled other invalid inputs. I therefore kept the expectations and recorded the failures as application problems. Bug 1 has two failing inputs, which explains the three failure events. These results showed that the initial suite could already detect bugs, but they did not tell me which important behavior it had missed. [2]

To choose the first improvement, I looked at the per-file coverage and the sample data. The overall 90.80% line coverage hid a much weaker area: `pdf_tools.py` had 74.77% line coverage and only 59.62% branch coverage. The review also showed that the sample PDFs already supplied title and author information. A successful upload of those samples gave little evidence about finding missing metadata from the page text.

This combination made PDF handling the first priority. It linked a measured gap to a specific weakness in the inputs, so I could give Codex a more useful request than simply asking for additional tests. The review also noted limits in error-message checks and a homepage test that only confirmed HTML delivery. Those observations left a separate question about how much the checks actually proved. I focused the first round on the PDF gap and returned to that question in round 2.

The review found no incorrect expected results. Some new tests overlapped with Homework 1 tests, which remained outside the comparison. Similar upload tests checked different outcomes, such as successful storage, duplicate handling, and rejection without saved files; no tests were removed as duplicates.

3. Iterative Test Improvement

Round 1: change what the PDF samples could reveal

I used the initial review to ask for file-size, page-count, and text-length boundaries, PDFs without supplied metadata, and failures during optional AI metadata extraction. I also made the expected effect of a rejected upload explicit:

> For rejected uploads, verify that no paper record or orphaned PDF file is created.

The reason for this instruction was that an error response alone would not show whether the failed request had left unwanted data behind. Similarly, removing the supplied PDF metadata would make the app use the extraction path that the original samples had avoided.

Codex retained the initial 37 methods and added eight. It tested the app's 20 MiB limit and one byte above it, 0/1/150/151-page PDFs, and text just below, at, and above one million characters. It also checked that a failed AI metadata request still allowed the app to save the PDF with its locally extracted information.

The README did not define the unit in `20 MB` more precisely, so these boundary tests used the app's current 20 MiB setting. This was a stated assumption in the round 1 notes.

The run passed 42 of 45 methods. All 35 previously passing methods still passed. Coverage rose to 96.74% for lines and 86.78% for branches, with the PDF module reaching 98.20% and 88.46%. However, one new method failed, so the pass rate had fallen. I needed to inspect that result before treating the round as successful or unsuccessful. [3]

Bug 3: a title continuation was saved as an author. The new sample had a two-line, 24-point title above a separate 12-point author line. The app returned:

```
Title:    Reliable Storage for
          Research Libraries
Expected: ['Dana Tester', 'Evan Researcher']
Actual:   ['Research Libraries', 'Dana Tester', 'Evan Researcher']
```

The detecting method in `test_pdf_round1.py` was:

```
test_multiline_prominent_title_is_not_misidentified_as_an_author
```

The sample's heading and author line had clear roles, and the wrong author appeared in both the response and the database. That supported treating it as an application mistake. The new failure also explained the lower pass rate: the added test had exposed a problem that the earlier samples could not reveal. I kept the check and counted the round as an improvement in bug detection. It did not establish that all PDF layouts would be handled correctly.

Round 2: follow the remaining gaps and inspect the checks

After reviewing round 1, I changed the focus. PDF coverage had improved considerably, while `providers.py` still had only 76.09% branch coverage. The round 1 review also continued to note limits in the existing checks. I used these findings to shape the second prompt: exercise the remaining service behavior and ask Codex to review what the tests actually checked.

I pointed Codex to the actual round 1 `coverage.json` and `experiment-notes.txt`. The feedback included:

> PDF handling improved substantially, while providers.py still has 76.09% branch coverage.

It also asked:

> Also review existing provider and error-handling assertions. Strengthen any that could pass despite incorrect results or state, and explain concrete examples.

This led to ten new methods. For example, the initial cache test repeated a search immediately. That could show reuse, but could not establish whether an expired result was refreshed or whether different searches got their own results. The new tests checked the ten-minute boundary and different query, page, and sort values. A controlled clock allowed timing checks without real waiting. Download tests checked that interruptions and rejected transfers preserved the saved paper and left no partial file.

That review led Codex to strengthen four existing methods. For example, it found that the earlier retry test ended by counting answers. A successful response and a count of one would still allow the wrong text to be saved:

```
self.assertEqual(len(self.detail(paper['id'])['analyses']), 1)
```

The revised test added these selected checks:

```
analysis = succeeded.json['analysis']
self.assertEqual(analysis['answer'], answer)
self.assertEqual(analysis['kind'], 'summary')
```

Here, `answer` is the known sample response. These checks require the expected text and task. Codex also found that the timeout test started with an empty library, so it changed the setup to save a paper first and verify that it survived. This is how the request to review existing checks led to concrete changes, even where code was already covered. [4]

The result was 52 passes out of 55 methods. All ten additions and four strengthened methods passed, while the same three application bugs still produced four failure events. Service-code coverage reached 98.46% for lines and 95.65% for branches. I interpreted this as evidence of better coverage and more specific checks. The observed bug count stayed at three, so this round did not demonstrate another increase in bugs found.

4. Final Evaluation

The two rounds answered different problems raised by the preceding output. The first changed the PDF inputs and revealed an additional bug. The second used the remaining coverage gaps and weak checks to examine service behavior more closely. The table shows their results on the same application code; the bug column counts distinct bugs found so far.

Suite	Passed / total	Line coverage	Branch coverage	Bugs found
Initial	35 / 37	90.80%	77.59%	2
Round 1	42 / 45	96.74%	86.78%	3
Round 2	52 / 55	98.47%	91.95%	3

The final suite covered 514/522 lines and 160/174 branch outcomes. All stages had zero test errors and zero skipped tests. The final saved output ended with:

```
Ran 55 tests in 13.357s

FAILED (failures=4)
```

The per-file results show why the focus moved from PDFs to services. Each value below is line coverage / branch coverage, in percent.

File	Initial	Round 1	Round 2
`app.py`	97.57 / 91.89	98.38 / 91.89	98.38 / 91.89
`database.py`	100 / 100	100 / 100	100 / 100
`pdf_tools.py`	74.77 / 59.62	98.20 / 88.46	98.20 / 88.46
`providers.py`	89.23 / 73.91	91.54 / 76.09	98.46 / 95.65

Coverage alone does not capture the whole improvement. Checking the saved answer more carefully can improve a test even when it executes the same lines. Conversely, a higher pass count does not mean the app was repaired: all previously passing methods continued to pass, more methods were added, and the known bugs remained.

The final review found no incorrect expected results, but eight lines and fourteen branch outcomes were still uncovered. They include deleting a paper while a model answer is being generated, rejecting unusually complex PDF data, and removing an old search result when the cache is full. I kept these as limits of the result rather than treating the high coverage as proof that the application was fully tested.

5. Reflection

I learned to use pass/fail counts as a starting point for review. The new PDF failure was useful because it could be traced to a clearly described sample. The passing retry test still needed work because its last check could miss the wrong answer. These examples made inspecting the inputs and checks as important as reading the coverage report.

Codex was effective at turning focused feedback into executable tests. Its first version was less effective at choosing difficult PDF samples and checking saved content. I would make sample data and expected results clearer in the first request, then continue using actual results to choose the next step. The useful input to the agent was our explanation of a weakness and the behavior we wanted the test to verify.

The use of fixed responses and a controlled clock reduced dependence on real services and timing. We did not run enough repetitions to measure how often a test might fail only sometimes. The experiment also did not test browser interaction, actual model answer quality, complicated PDF layouts, simultaneous requests, or crashes. Those limits follow from the kinds of tests used.

Codex helped review the outputs and prepare the prompts, tests, and report. I used that analysis to direct the two improvement rounds. I did not manually edit the tests or fix the application during this comparison.

6. GitHub Repository

Repository: `https://github.com/VichyTong/CS593-ASE`.

The final tests are in `solutions/hw2/round2_stage/tests/`. The report and LaTeX source are in `solutions/hw2/docs/`. Full prompts, review notes, and recorded results are saved alongside each stage.

From the repository root, use Python 3.11 or newer:

```
python3 -m venv /tmp/papertrail-hw2-venv
TASK_PY=/tmp/papertrail-hw2-venv/bin/python
"$TASK_PY" -m pip install -r \
  solutions/hw2/round2_stage/requirements-test.txt
"$TASK_PY" -B solutions/hw2/round2_stage/run_round2.py \
  --output /tmp/papertrail-hw2-final-run
```

Use a new output directory each time and keep the recorded stage files together. The runner checks those files before starting and saves test output, `results.json`, `coverage.json`, and HTML coverage. Use the separate line and branch values in `results.json` to compare with this report.

On the recorded application, the expected result is 52 passed methods, three failed methods, and four failure events. Exit code 1 reflects the three known bugs. Temporary data and fixed service responses allow the tests to run without an API key.

[1] `solutions/hw1/README.md`: application requirements.

[2] `solutions/hw2/initial_stage/`: initial request, review notes, and `evidence/initial-only/`.

[3] `solutions/hw2/round1_stage/`: first feedback, review notes, and `evidence/round1-only/`.

[4] `solutions/hw2/round2_stage/`: second feedback, review notes, `evidence/round2-only/`, `evidence/comparison.json`, and `evidence/retained-test-changes.diff`.